# Ciencia de Datos para Economía y Negocios - FCE-UBA

## Clase 3 — Tidyverse: lectura de datos y verbos principales

Nicolás Sidicaro

# Hoja de ruta de hoy

1. Parte I: Filosofía tidy
2. Parte II: Lectura de datos
3. Parte III: El operador pipe
4. Parte IV: Los 5 verbos de dplyr
5. Parte V: Combinando verbos
6. Parte VI: Resumen y cierre

# Parte I: Filosofía tidy

# ¿Qué es el Tidyverse?

El **Tidyverse** es una colección de paquetes de R diseñados para ciencia de datos.

Comparten una misma filosofía de diseño, gramática y estructura de datos.

```r
install.packages("tidyverse")  # instalar (una sola vez)
library(tidyverse)             # cargar en cada sesión
```

Paquetes principales que incluye:

- **dplyr**: manipulación de datos (los "verbos")
- **readr**: lectura de archivos CSV, TSV, etc.
- **tidyr**: reestructuración de datos
- **ggplot2**: visualización (lo veremos más adelante)
- **tibble**, **stringr**, **forcats**, **purrr**...

# Los 3 principios de los datos "tidy"

Hadley Wickham (2014) definió tres reglas para que un dataset esté en formato **tidy**:

**Regla 1**

Cada **variable** ocupa una **columna**.

**Regla 2**

Cada **observación** ocupa una **fila**.

**Regla 3**

Cada **valor** ocupa una **celda**.

¿Por qué importa? Porque **todos los verbos de dplyr asumen datos tidy**. Si los datos no están en este formato, primero hay que transformarlos.

# Ejemplo: tidy vs. no tidy

**No tidy (formato ancho)**

| País | 2020 | 2021 | 2022 |
|---|---|---|---|
| ARG | 8,571 | 10,636 | 13,650 |
| BRA | 6,797 | 7,519 | 8,918 |
| CHL | 13,231 | 16,265 | 16,800 |

Los años son **valores**, no variables.

**Tidy (formato largo)**

| País | Año | PBI_pc |
|---|---|---|
| ARG | 2020 | 8,571 |
| ARG | 2021 | 10,636 |
| ARG | 2022 | 13,650 |
| BRA | 2020 | 6,797 |
| ... | ... | ... |

Cada fila = una observación (país-año).

La conversión entre formatos se hace con `pivot_longer()` y `pivot_wider()` (lo veremos en la próxima clase).

# Parte II: Lectura de datos

# Leer archivos CSV: `read_csv()`

La función `read_csv()` (del paquete **readr**) lee archivos separados por comas:

```r
library(tidyverse)

# Leer desde un archivo local
datos <- read_csv("datos/pbi_latam.csv")

# Leer desde una URL
datos <- read_csv("https://ejemplo.com/datos.csv")
```

Ventajas sobre `read.csv()` de R base:

- Más rápido para archivos grandes
- Detecta tipos de columna automáticamente
- Devuelve un **tibble** (versión mejorada de data.frame)
- No convierte strings a factores por defecto

# Leer archivos Excel: `read_excel()`

Para archivos `.xlsx` o `.xls` usamos el paquete **readxl**:

```r
library(readxl)

# Leer la primera hoja
datos <- read_excel("datos/indicadores.xlsx")

# Especificar hoja y rango
datos <- read_excel("datos/indicadores.xlsx",
                    sheet = "PBI",
                    range = "A1:D50")

# Saltar filas de encabezado
datos <- read_excel("datos/indicadores.xlsx",
                    skip = 3)
```

# Otros formatos: `read_delim()` y más

`read_delim()` es la función general para cualquier delimitador:

```r
# Archivos separados por punto y coma (común en español)
datos <- read_delim("archivo.csv", delim = ";")

# Archivos separados por tabulación
datos <- read_delim("archivo.tsv", delim = "\t")

# Equivalente directo para TSV
datos <- read_tsv("archivo.tsv")
```

**¿Por qué punto y coma?**

En países donde el separador decimal es la coma (como Argentina), los CSV suelen usar `;` como separador de columnas. Si abren un CSV del INDEC y ven todo pegado en una columna, prueben con `read_delim()` y `delim = ";"`.

# Argumentos comunes de lectura

Todas las funciones de lectura comparten argumentos útiles:

| Argumento | Función | Ejemplo |
|---|---|---|
| `col_names` | Usar primera fila como encabezado | `col_names = FALSE` |
| `skip` | Saltar *n* filas al inicio | `skip = 2` |
| `na` | Valores que representan NA | `na = c("", "NA", "-")` |
| `col_types` | Forzar tipos de columna | `col_types = "ccnd"` |
| `locale` | Config. regional (decimales, encoding) | `locale = locale(decimal_mark = ",")` |
| `n_max` | Leer solo las primeras *n* filas | `n_max = 100` |

```r
# Ejemplo combinando argumentos
datos <- read_csv("indec_ipc.csv",
                  skip = 4,
                  na = c("", "-", "///"),
```

# Parte III: El operador pipe

# El problema: código anidado

Sin pipe, para encadenar operaciones hay que **anidar funciones**:

```
# Queremos: leer datos, filtrar, seleccionar y ordenar
# Sin pipe: se lee de adentro hacia afuera
arrange(select(filter(datos, pais == "ARG"),
               anio, pbi_pc), desc(pbi_pc))
```

Problemas:

- Se lee de **adentro hacia afuera** (poco intuitivo)
- Difícil de modificar y depurar
- Los paréntesis se acumulan rápidamente

La alternativa era crear **objetos intermedios**, pero genera nombres innecesarios:

```
datos_filtrados  <- filter(datos, pais == "ARG")
datos_select     <- select(datos_filtrados, anio, pbi_pc)
```

# La solución: el pipe |>

El pipe toma el resultado de la izquierda y lo pasa como **primer argumento** de la función de la derecha:

```
# Con pipe: se lee de arriba hacia abajo
datos |>
  filter(pais == "ARG") |>
  select(anio, pbi_pc) |>
  arrange(desc(pbi_pc))
```

Se lee como una secuencia de instrucciones: "*Tomá* `datos`, ***después*** *filtrá*, ***después*** *seleccioná*, ***después*** *ordená*."

**Atajo de teclado**

En RStudio: **Ctrl + Shift + M** (Windows/Linux) o **Cmd + Shift + M** (Mac).

# |> nativo vs. %>% de magrittr

**|> (R $\geq$ 4.1)**

- Viene con R, no necesita paquetes
- Más rápido (no hay overhead)
- Es el estándar recomendado hoy
- Limitación: no tiene `.` como placeholder

**%>% (magrittr/tidyverse)**

- Requiere `library(tidyverse)` o `library(magrittr)`
- Permite usar `.` para posicionar el argumento
- Fue el pipe original del Tidyverse
- Sigue funcionando, pero ya no es el preferido

Para esta materia vamos a usar |>. Si encuentran %>% en código de internet, sepan que hace **exactamente lo mismo** en la enorme mayoría de los casos.

# Parte IV: Los 5 verbos de dplyr

# Nuestro dataset de trabajo

Vamos a trabajar con datos económicos de América Latina. Primero los creamos:

```
library(tidyverse)

latam <- tibble(
  pais = rep(c("ARG","BRA","CHL","COL","MEX","PER","URY"), each = 3),
  anio = rep(2020:2022, times = 7),
  pbi_pc = c(8571,10636,13650, 6797,7519,8918, 13231,16265,16800,
             5334,6131,6624, 8347,10046,11091, 6127,6692,7126,
             15438,17313,20795),
  inflacion = c(36.1,50.9,72.4, 4.5,10.1,5.8, 3.0,7.2,11.6,
                1.6,5.6,10.2, 3.2,7.4,7.9, 2.0,6.4,8.5,
                9.8,8.0,8.3),
  poblacion_mill = c(45.4,45.8,46.2, 213.2,214.3,215.3,
                     19.5,19.5,19.6, 51.3,51.5,51.9,
                     128.9,129.9,130.1, 33.0,33.4,33.7,
```

# Vista rápida de los datos

```r
glimpse(latam)
```

```
Rows: 21
Columns: 6
$ pais           <chr> "ARG", "ARG", "ARG", "BRA", "BRA", "BRA", "CHL", "CHL"
$ anio           <int> 2020, 2021, 2022, 2020, 2021, 2022, 2020, 2021, 2022, 2
$ pbi_pc         <dbl> 8571, 10636, 13650, 6797, 7519, 8918, 13231, 16265, 168
$ inflacion      <dbl> 36.1, 50.9, 72.4, 4.5, 10.1, 5.8, 3.0, 7.2, 11.6, 1.6,
$ poblacion_mill <dbl> 45.4, 45.8, 46.2, 213.2, 214.3, 215.3, 19.5, 19.5, 19.6
$ region         <chr> "Cono Sur", "Cono Sur", "Cono Sur", "Andina", "Andina"
```

Funciones útiles para explorar un dataset:

- `glimpse()`: estructura compacta (tipo y primeros valores)
- `head()` / `tail()`: primeras o últimas filas
- `nrow()`, `ncol()`, `dim()`: dimensiones
- `names()`: nombres de columnas

# select() — Elegir columnas

select() permite quedarnos solo con las columnas que necesitamos:

```r
# Seleccionar columnas por nombre
latam |>
  select(pais, anio, pbi_pc)
```

```
# A tibble: 21 x 3
   pais   anio pbi_pc
   <chr> <int>  <dbl>
 1 ARG    2020   8571
 2 ARG    2021  10636
 3 ARG    2022  13650
 4 BRA    2020   6797
 5 BRA    2021   7519
 6 BRA    2022   8918
 7 CHL    2020  13231
```

# select() — Funciones auxiliares

```r
# Excluir columnas con -
latam |> select(-region, -poblacion_mill)
```

```
# A tibble: 21 x 4
   pais   anio pbi_pc inflacion
   <chr> <int>  <dbl>     <dbl>
 1 ARG    2020   8571      36.1
 2 ARG    2021  10636      50.9
 3 ARG    2022  13650      72.4
 4 BRA    2020   6797       4.5
 5 BRA    2021   7519      10.1
 6 BRA    2022   8918       5.8
 7 CHL    2020  13231       3
 8 CHL    2021  16265       7.2
 9 CHL    2022  16800      11.6
10 COL    2020   5334       1.6
# i 11 more rows
```

```r
# Seleccionar un rango de columnas con :
latam |> select(pais:pbi_pc)
```

# `filter()` — Filtrar filas

`filter()` selecciona filas que cumplen una condición lógica:

```r
# Filtrar un país
latam |>
  filter(pais == "ARG")
```

```
# A tibble: 3 x 6
  pais   anio pbi_pc inflacion poblacion_mill region
  <chr> <int>  <dbl>     <dbl>          <dbl> <chr>
1 ARG    2020   8571      36.1           45.4 Cono Sur
2 ARG    2021  10636      50.9           45.8 Cono Sur
3 ARG    2022  13650      72.4           46.2 Cono Sur
```

Operadores lógicos disponibles: `==`, `!=`, `>`, `<`, `>=`, `<=`, `&` (y), `|` (o), `!` (negación), `%in%` (pertenencia).

# filter() — Condiciones múltiples

```r
# Países del Cono Sur con inflación > 10%
latam |>
  filter(region == "Cono Sur", inflacion > 10)
```

```
# A tibble: 4 x 6
  pais   anio pbi_pc inflacion poblacion_mill region
  <chr> <int>  <dbl>     <dbl>          <dbl> <chr>
1 ARG    2020   8571      36.1           45.4 Cono Sur
2 ARG    2021  10636      50.9           45.8 Cono Sur
3 ARG    2022  13650      72.4           46.2 Cono Sur
4 CHL    2022  16800      11.6           19.6 Cono Sur
```

# Operador %in%

```r
# Varios países con %in%
latam |>
  filter(pais %in% c("ARG", "BRA", "MEX"), anio == 2022)
```

```
# A tibble: 3 x 6
  pais   anio pbi_pc inflacion poblacion_mill region
  <chr> <int>  <dbl>     <dbl>          <dbl> <chr>
1 ARG    2022  13650      72.4           46.2 Cono Sur
2 BRA    2022   8918       5.8          215.  Andina
3 MEX    2022  11091       7.9          130.  Norteamérica
```

# `mutate()` — Crear o modificar columnas

`mutate()` agrega nuevas columnas o modifica las existentes:

```
latam |>
  mutate(
    pbi_total_mill = pbi_pc * poblacion_mill,
    inflacion_dec  = inflacion / 100
  ) |>
  select(pais, anio, pbi_total_mill, inflacion_dec)
```

```
# A tibble: 21 x 4
   pais   anio pbi_total_mill inflacion_dec
   <chr> <int>          <dbl>         <dbl>
 1 ARG    2020        389123.         0.361
 2 ARG    2021        487129.         0.509
 3 ARG    2022        630630          0.724
 4 BRA    2020       1449120.         0.045
```

# mutate() — Operaciones comunes

```r
latam |>
  filter(pais == "ARG") |>
  mutate(
    log_pbi     = round(log(pbi_pc), 2),
    infl_alta   = ifelse(inflacion > 20, "Sí", "No"),
    pbi_en_miles = round(pbi_pc / 1000, 1)
  ) |>
  select(pais, anio, log_pbi, infl_alta, pbi_en_miles)
```

```
# A tibble: 3 x 5
  pais   anio log_pbi infl_alta pbi_en_miles
  <chr> <int>   <dbl> <chr>            <dbl>
1 ARG    2020    9.06 Sí                 8.6
2 ARG    2021    9.27 Sí                10.6
3 ARG    2022    9.52 Sí                13.7
```

# arrange() — Ordenar filas

arrange() ordena las filas según una o más columnas:

```r
# Ordenar por PBI per cápita (ascendente, por defecto)
latam |>
  filter(anio == 2022) |>
  arrange(pbi_pc)
```

```
# A tibble: 7 x 6
  pais   anio pbi_pc inflacion poblacion_mill region
  <chr> <int>  <dbl>     <dbl>          <dbl> <chr>
1 COL    2022   6624      10.2           51.9 Andina
2 PER    2022   7126       8.5           33.7 Andina
3 BRA    2022   8918       5.8          215.  Andina
4 MEX    2022  11091       7.9          130.  Norteamérica
5 ARG    2022  13650      72.4           46.2 Cono Sur
6 CHL    2022  16800      11.6           19.6 Cono Sur
```

# arrange() — Orden descendente

```r
# Orden descendente con desc()
latam |>
  filter(anio == 2022) |>
  arrange(desc(inflacion))
```

```
# A tibble: 7 x 6
  pais   anio pbi_pc inflacion poblacion_mill region
  <chr> <int>  <dbl>     <dbl>          <dbl> <chr>
1 ARG    2022  13650      72.4           46.2 Cono Sur
2 CHL    2022  16800      11.6           19.6 Cono Sur
3 COL    2022   6624      10.2           51.9 Andina
4 PER    2022   7126       8.5           33.7 Andina
5 URY    2022  20795       8.3            3.6 Cono Sur
6 MEX    2022  11091       7.9          130.  Norteamérica
7 BRA    2022   8918       5.8          215.  Andina
```

# summarise() — Resumir datos

summarise() colapsa el dataframe a una fila con estadísticas resumen:

```
latam |>
  filter(anio == 2022) |>
  summarise(
    pbi_promedio    = mean(pbi_pc),
    pbi_mediana     = median(pbi_pc),
    infl_max        = max(inflacion),
    n_paises        = n()
  )
```

```
# A tibble: 1 x 4
  pbi_promedio pbi_mediana infl_max n_paises
         <dbl>       <dbl>    <dbl>    <int>
1       12143.       11091     72.4        7
```

Funciones de resumen comunes: mean(), median(), sd(), min(), max(), sum(), n()

# summarise() + group_by() — Resumen por grupos

La gracia aparece al combinar summarise() con group_by():

```
latam |>
  filter(anio == 2022) |>
  group_by(region) |>
  summarise(
    pbi_promedio = round(mean(pbi_pc), 0),
    infl_promedio = round(mean(inflacion), 1),
    n_paises = n()
  )
```

```
# A tibble: 3 x 4
  region       pbi_promedio infl_promedio n_paises
  <chr>               <dbl>         <dbl>    <int>
1 Andina               7556           8.2        3
2 Cono Sur            17082          30.8        3
```

# Parte V: Combinando verbos

# Encadenando operaciones

La verdadera potencia del Tidyverse aparece al **encadenar verbos con el pipe**:

```
# ¿Cuál fue el crecimiento del PBI per cápita 2020-2022?
latam |>
  filter(anio %in% c(2020, 2022)) |>
  select(pais, anio, pbi_pc) |>
  pivot_wider(names_from = anio, values_from = pbi_pc,
              names_prefix = "pbi_") |>
  mutate(
    crecimiento_pct = round((pbi_2022 / pbi_2020 - 1) * 100, 1)
  ) |>
  arrange(desc(crecimiento_pct))
```

```
# A tibble: 7 x 4
  pais  pbi_2020 pbi_2022 crecimiento_pct
  <chr>    <dbl>    <dbl>           <dbl>
```

# Ejercicio guiado: ranking de inflación

Paso a paso, construyamos un ranking:

```r
# Top 5 observaciones con mayor inflación
latam |>                                  # 1. Partir del dataset
  select(pais, anio, inflacion) |>        # 2. Quedarnos con lo relevante
  arrange(desc(inflacion)) |>             # 3. Ordenar de mayor a menor
  head(5)                                 # 4. Tomar las primeras 5
```

```
# A tibble: 5 x 3
  pais   anio inflacion
  <chr> <int>     <dbl>
1 ARG    2022      72.4
2 ARG    2021      50.9
3 ARG    2020      36.1
4 CHL    2022      11.6
5 COL    2022      10.2
```

# Ejercicio: inflación promedio por país

```r
latam |>
  group_by(pais) |>
  summarise(
    infl_promedio = round(mean(inflacion), 1),
    infl_desvio   = round(sd(inflacion), 1),
    anios         = n()
  ) |>
  arrange(desc(infl_promedio))
```

```
# A tibble: 7 x 4
  pais  infl_promedio infl_desvio anios
  <chr>         <dbl>       <dbl> <int>
1 ARG            53.1        18.3     3
2 URY             8.7         1       3
3 CHL             7.3         4.3     3
4 BRA             6.8         2.9     3
```

# Parte VI: Resumen y cierre

# Los verbos de hoy

| Verbo | Acción |
|---|---|
| `select()` | Elegir **columnas** |
| `filter()` | Filtrar **filas** por condición |
| `mutate()` | Crear o modificar **columnas** |
| `arrange()` | Ordenar **filas** |
| `summarise()` | Calcular **estadísticas resumen** |
| `group_by()` | Agrupar para operar por categoría |
| `\|>` | **Pipe**: encadena operaciones |

Todos los verbos siguen la misma lógica: **el primer argumento es el dataframe** y el resultado es otro dataframe. Esto permite encadenarlos con el pipe.

# Próxima clase

## Clase 4: Datos tabulares — agrupamiento, pivots y joins

- `group_by()` + `summarise()` en profundidad
- `pivot_longer()` y `pivot_wider()`
- `if_else()`, `case_when()`
- Modelo relacional: claves primarias y foráneas
- Tipos de joins y cuándo usar cada uno

**Bibliografía de la clase de hoy:**


- Wickham, Çetinkaya-Rundel & Grolemund (2023). *R for Data Science*, 2da ed. Cap. 3 (Data transformation) y Cap. 7 (Data import). Disponible en: r4ds.hadley.nz o link de bibliografía del Github del curso